

Google Maps Platform

para Entregas de Pescado

APIS • ROTEAMENTO • RASTREAMENTO • CASO PRÁTICO

Routes API

Geocoding API

Distance Matrix

Realtime GPS

Geofencing

Visão Geral: Ecossistema de APIs

O Google Maps Platform é organizado em **três famílias de produtos**. Para o módulo de roteirização do PescApp, utilizamos as três.

MAPS

Renderização visual de mapas interativos no app do cliente e no painel de gestão

ROUTES

Cálculo de rotas, otimização de múltiplas paradas e estimativa de chegada (ETA)

PLACES & GEOCODING

Conversão de endereços textuais em coordenadas geográficas para o checkout

 Todas as APIs exigem uma **API Key** com as permissões corretas ativadas no Google Cloud Console. Uma única chave pode habilitar múltiplas APIs do mesmo projeto.

Arquitetura de Integração

Como cada módulo do PescApp consome as APIs:

Pescador (WhatsApp Bot) — sem uso direto de Maps

App Cliente → Maps JS API + Geocoding + Rastreamento RT

Backend (Node.js) → Routes API + Distance Matrix + Geofencing

Painel Gestão → Maps JS API + Fleet Tracking

PRINCÍPIO DE SEGURANÇA

Chamadas com dados sensíveis (frete, despacho) devem ser feitas no **backend** — nunca expor a API Key no código frontend.

RF02 – Geocodificação de Endereços

Converte o endereço do checkout em `{lat, lng}` para plotagem e cálculo de frete.

```
// backend/services/geocoding.service.js
const axios = require('axios');

async function geocodeAddress(address) {
  const { data } = await axios.get(
    'https://maps.googleapis.com/maps/api/geocode/json',
    {
      params: {
        address: `${address}, Manguinhos, Rio de Janeiro, BR`,
        key: process.env.GOOGLE_MAPS_API_KEY,
      },
    }
  );

  if (data.status !== 'OK') throw new Error(`Geocoding falhou: ${data.status}`);

  const { lat, lng } = data.results[0].geometry.location;
  return {
    lat,
    lng,
    formattedAddress: data.results[0].formatted_address,
  };
}

module.exports = { geocodeAddress };
```

RF05 – Cálculo Dinâmico de Frete

Calcula o valor da entrega via Distance Matrix com bloqueio por raio máximo (RN01).

```
// backend/services/freight.service.js
const TARIFF = { BASE_FEE: 3.00, PRICE_PER_KM: 1.50, FREE_KM: 2, MAX_KM: 15 };

async function calculateFreight(origin, destination) {
  const { data } = await axios.get(
    'https://maps.googleapis.com/maps/api/distancematrix/json',
    {
      params: {
        origins: `${origin.lat},${origin.lng}`,
        destinations: `${destination.lat},${destination.lng}`,
        mode: 'driving',
        key: process.env.GOOGLE_MAPS_API_KEY,
      },
    },
  );

  const el = data.rows[0].elements[0];
  if (el.status !== 'OK') throw new Error('Rota não encontrada');

  const distKm = el.distance.value / 1000;
  const durMin = Math.ceil(el.duration.value / 60);

  // RN01 - Bloquear pedidos além do raio de frescor (15 km)
  if (distKm > TARIFF.MAX_KM)
    throw new Error(`Fora da área: ${distKm.toFixed(1)}km > ${TARIFF.MAX_KM}km`);

  const extraKm = Math.max(0, distKm - TARIFF.FREE_KM);
  const freight = parseFloat((TARIFF.BASE_FEE + extraKm * TARIFF.PRICE_PER_KM).toFixed(2));

  return { distKm: distKm.toFixed(2), durMin, freight };
}
```

RF01 — Rota Otimizada com Múltiplas Paradas

Routes API v2 reordena as paradas automaticamente; RNo2 pré-ordena por perecibilidade.

```
// backend/services/routing.service.js
async function computeOptimizedRoute(origin, waypoints, destination) {
  // RNo2 - itens mais perecíveis entram primeiro na lista
  const sorted = [...waypoints].sort((a, b) => b.perecibilityScore - a.perecibilityScore);

  const { data } = await axios.post(
    'https://routes.googleapis.com/directions/v2:computeRoutes',
    {
      origin:      { location: { latLng: origin } },
      destination: { location: { latLng: destination } },
      intermediates: sorted.map(wp => ({ location: { latLng: wp } })),
      travelMode:   'TWO_WHEELER',
      optimizeWaypointOrder: true,
      routingPreference: 'TRAFFIC_AWARE',
      languageCode:   'pt-BR',
    },
    {
      headers: {
        'X-Goog-API-Key': process.env.GOOGLE_MAPS_API_KEY,
        'X-Goog-FieldMask': 'routes.duration,routes.distanceMeters,routes.optimizedIntermediateWaypointIndex',
      },
    }
  );

  const r = data.routes[0];
  return {
    totalKm:      (r.distanceMeters / 1000).toFixed(2),
    totalMin:      Math.ceil(parseInt(r.duration) / 60),
    optimizedOrder: r.optimizedIntermediateWaypointIndex,
  };
}
```

RFo4 – Rastreamento em Tempo Real

App do motoboy envia posição a cada 2 s (RNFo1). Cliente consome via WebSocket.

```
// app-entregador – React Native (Expo)
import * as Location from 'expo-location';
import { socket } from '../services/socket';

export function useGPSTracker(orderId, isActive) {
  useEffect(() => {
    if (!isActive) return;
    let watcher;
    (async () => {
      const { status } = await Location.requestForegroundPermissionsAsync();
      if (status !== 'granted') return;
      // RNF01: intervalo ≤ 2 s | RNF02: precisão ≤ 15 m
      watcher = await Location.watchPositionAsync(
        { accuracy: Location.Accuracy.High, TimeInterval: 2000, distanceInterval: 5 },
        ({ coords }) => {
          if (coords.accuracy > 15) return;
          socket.emit('gps:update', { orderId, lat: coords.latitude, lng: coords.longitude });
        }
      );
    })();
    return () => watcher?.remove();
  }, [isActive, orderId]);
}
```

RF06 — Geofencing & RNo4 — Anti-Fraude

Dois usos da fórmula **Haversine** — sem custo de API, 100% no backend.

```
// Haversine: distância em metros entre dois pontos lat/lng
function haversineDistance({ lat: lat1, lng: lng1 }, { lat: lat2, lng: lng2 }) {
  const R = 6371e3, rad = d => d * Math.PI / 180;
  const dLat = rad(lat2 - lat1), dLng = rad(lng2 - lng1);
  const a = Math.sin(dLat/2)**2
    + Math.cos(rad(lat1)) * Math.cos(rad(lat2)) * Math.sin(dLng/2)**2;
  return R * 2 * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
}

// RF06 — push quando entregador entra no raio de 500 m
async function checkGeofence(coords, order) {
  if (haversineDistance(coords, order.destination) ≤ 500 && !order.geofenceNotified) {
    await order.update({ geofenceNotified: true });
    await sendPush(order.clientFcmToken, { title: '🐟 Chegando!', body: 'Menos de 500m!' });
  }
}

// RNo4 — entregador só confirma "Entregue" estando a ≤ 50 m do destino
async function validateDelivery(deviceCoords, order) {
  const dist = haversineDistance(deviceCoords, order.destination);
  if (dist > 50) throw new Error(`GPS fora do raio: ${dist.toFixed(0)}m (máx 50m)`);
  await api.patch(`/orders/${order.id}/status`, { status: 'DELIVERED', proofLocation: deviceCoords });
}
```


RFo7 – Despacho por Proximidade

Seleciona o motoboy online mais próximo do porto no momento do pedido.

```
// backend/services/dispatch.service.js
async function assignNearestDeliverer(order) {
  const available = await Deliverer.findAll({
    where: { status: 'ONLINE', activeOrders: { [Op.lt]: 3 } },
  });

  const ranked = available
    .map(d => ({
      ...d.toJSON(),
      dist: haversineDistance(
        { lat: d.currentLat, lng: d.currentLng },
        { lat: order.pickupLat, lng: order.pickupLng }
      ),
    }))
    .sort((a, b) => a.dist - b.dist);

  const nearest = ranked[0];

  if (!nearest || nearest.dist > 5000) {
    await order.update({ status: 'WAITING_DELIVERER' });
    await notifyAdmin(order);
    return null;
  }

  await order.update({ delivererId: nearest.id, status: 'ASSIGNED' });
  await sendPush(nearest.fcmToken, {
    title: '🚚 Novo pedido!',
    body: `Coleta a ${nearest.dist < 1000
      ? `${nearest.dist.toFixed(0)}m`
      : `${(nearest.dist/1000).toFixed(1)}km`} - ${order.fishSpecies}`,
  });
  return nearest;
}
```

Caso de Teste: Pedido Completo

Simulação end-to-end — 3 kg de robalo, Porto de Mangueiros → Laranjeiras.

```
// Entrada
const order = {
  clientAddress: "Rua Alice, 10, Laranjeiras, RJ",
  pickupCoords: { lat: -22.8621, lng: -43.2298 },
  items: [{ species: 'Robalo', weight: 3, cut: 'filé' }],
};
```

```
// Saída esperada
{
  "geocoding": { "lat": -22.9285, "lng": -43.1826 },
  "freight": { "distKm": "11.40", "durMin": 28, "value": 19.10 },
  "deliverer": { "name": "Carlos Moto", "dist": "620m" }
}
```

① GEOCODING API

Endereço → **lat: -22.9285** **lng: -43.1826**

② DISTANCE MATRIX

11,4 km → R\$ 19,10 · 28 min estimado

③ RN01 — RAI0 OK

11,4 km < 15 km  entrega liberada

④ RF07 — DESPACHO

Carlos Moto a 620 m do porto · selecionado

⑤ GPS LOOP 2 S

Rastreamento ativo · geofence 500 m armado

⑥ RN04 — ENTREGA

GPS valida ≤ 50 m → status **DELIVERED**

Custos e Boas Práticas

SKUS – FAIXA GRATUITA/MÊS

API	GRATUITO	EXCEDENTE
Maps JS	28.000 loads	\$7 / 1k
Geocoding	40.000 req	\$5 / 1k
Distance Matrix	100.000 elem	\$5 / 1k
Routes API	10.000 req	\$10 / 1k
Maps Static	100.000 req	\$2 / 1k

* Google concede \$200/mês de crédito. Com 25 pescadores no MVP, custo estimado: \$0/mês.

ECONOMIA E SEGURANÇA

CACHE DE GEOCODING

Salvar `lat/lng` no banco após a 1ª chamada — evita reprocessamento

DISTANCE MATRIX NO BACKEND

Nunca chamar direto do app — centralizar e cachear por par origem/destino

RESTRIÇÃO DE API KEY

Limitar a chave frontend ao domínio via *HTTP Referrer* no Cloud Console

Resumo das Integrações

GEOCODING API

RFo2

Checkout → coordenadas para frete e mapa

DISTANCE MATRIX

RFo5

Cálculo dinâmico de frete por km + RNo1

ROUTES API V2

RFo1

Otimização multi-parada + RNo2 perecibilidade

MAPS JS API

RFo4

Mapa interativo de rastreamento em tempo real

GPS + HAVERSINE

RNo4

Anti-fraude na confirmação de entrega

GEOFENCING

RFo6

Push automático a 500 m do destino